



Database Management Systems (DBMS)

Complete Interview Q&A Guide — Basic to Advanced

Covers every DBMS concept from **Fundamentals, Relational Model, SQL, Normalization, Transactions, Indexing, Query Optimization, NoSQL, Distributed Databases**, and more.

□ Table of Contents

1. [DBMS Fundamentals](#)
 2. [Data Models](#)
 3. [Entity-Relationship \(ER\) Model](#)
 4. [Relational Model](#)
 5. [SQL — DDL, DML, DCL, TCL](#)
 6. [SQL — Queries, Joins & Subqueries](#)
 7. [SQL — Aggregate Functions, Views & Stored Procedures](#)
 8. [Normalization](#)
 9. [Transactions & ACID Properties](#)
 10. [Concurrency Control](#)
 11. [Deadlocks](#)
 12. [Indexing & Hashing](#)
 13. [Query Processing & Optimization](#)
 14. [File Organization & Storage](#)
 15. [Recovery & Crash Management](#)
 16. [Relational Algebra & Calculus](#)
 17. [NoSQL Databases](#)
 18. [Distributed Databases](#)
 19. [Advanced DBMS Concepts](#)
 20. [Database Design & Best Practices](#)
-

1. DBMS Fundamentals

Q1. What is a DBMS? How is it different from a file system?

A: A **Database Management System (DBMS)** is software that creates, manages, and provides controlled access to a structured collection of data. It acts as an interface between users/applications and the physical database.

DBMS vs File System:

Aspect	File System	DBMS
Data redundancy	High (duplicate files)	Controlled via normalization
Data inconsistency	Common	Avoided with integrity constraints
Data access	Manual, program-specific	Declarative queries (SQL)
Security	OS-level only	Fine-grained access control
Concurrency	No built-in support	Multi-user concurrency control
Recovery	Manual	Automatic crash recovery
Data independence	None	Physical + logical independence
Relationships	Not supported	First-class concept

Examples of DBMS: MySQL, PostgreSQL, Oracle, SQL Server, MongoDB, Redis.

Q2. What are the advantages of using a DBMS?

A:

- **Data Independence:** Changes to storage don't affect applications (physical independence); changes to logical schema don't affect views (logical independence).
 - **Reduced Redundancy:** Centralized data reduces duplication and inconsistency.
 - **Data Integrity:** Constraints (PK, FK, CHECK, UNIQUE) ensure valid data.
 - **Concurrency Control:** Multiple users can safely access data simultaneously.
 - **Security:** Authentication, authorization, and auditing at fine granularity.
 - **Crash Recovery:** Transaction logs enable restoration after failure.
 - **Query Language:** SQL allows complex data retrieval without knowing physical storage.
 - **Backup and Recovery:** Automated tools for data protection.
-

Q3. What are the types of DBMS?

A:

Type	Description	Examples
Relational (RDBMS)	Data in tables with rows/columns; SQL; ACID	MySQL, PostgreSQL, Oracle, SQL Server

Type	Description	Examples
Hierarchical	Tree structure; parent-child relationships	IBM IMS
Network	Graph structure; records linked via pointers	IDS, CA-IDMS
Object-Oriented	Objects as in OOP; complex types	ObjectDB, db4o
NoSQL	Non-relational; flexible schema; horizontal scaling	MongoDB, Cassandra, Redis, Neo4j
NewSQL	RDBMS with NoSQL scalability	Google Spanner, CockroachDB
Time-Series	Optimized for timestamped data	InfluxDB, TimescaleDB
Graph	Nodes and edges for relationship-heavy data	Neo4j, Amazon Neptune

Q4. What are the three levels of data abstraction in DBMS?

A: DBMS provides three layers of abstraction to hide complexity:

1. **Physical Level (Internal):** Lowest level. Describes *how* data is physically stored — file formats, index structures, storage blocks, compression. Managed by the DBMS storage engine.
2. **Logical Level (Conceptual):** Describes *what* data is stored and the relationships between entities — tables, columns, data types, constraints. This is the DBA's view.
3. **View Level (External):** Highest level. Describes *which part* of the database a specific user or application sees — customized views, hiding irrelevant or sensitive data.

This three-level architecture enables **data independence** — changes at a lower level don't affect higher levels.

Q5. What is data independence?

A:

- **Physical Data Independence:** The ability to change the physical storage structure (e.g., change file organization from sequential to indexed) without affecting the logical schema or applications. Most DBMS provide this.
- **Logical Data Independence:** The ability to change the logical schema (e.g., add a new table or column) without affecting existing views or applications. Harder to achieve fully.

Data independence is the main reason we use a DBMS over direct file access.

Q6. What is a schema vs an instance?

A:

- **Schema:** The structure or blueprint of the database — table names, column names, data types, constraints. Defined at design time. Rarely changes. Analogous to a class in OOP.
 - **Instance (Database State):** The actual data stored in the database at a particular moment in time. Changes constantly with insertions, updates, and deletions. Analogous to an object in OOP.
-

Q7. What are the roles in a database environment?

A:

- **DBA (Database Administrator):** Manages schema design, performance tuning, security, backups, and user privileges.
 - **Database Designer:** Creates the ER model and relational schema.
 - **Application Developer:** Writes programs that interact with the DBMS.
 - **End Users:** Casual users (forms/reports) and sophisticated users (SQL queries).
 - **DBMS Vendor:** Provides and maintains the DBMS software.
-

2. Data Models

Q8. What is a data model?

A: A data model defines the logical structure of a database — how data is organized, related, and constrained. It provides a formal way to describe data.

Categories:

- **Conceptual (High-level):** Close to how users perceive data. Entity-Relationship (ER) model.
 - **Logical (Implementation):** Defines structure in terms of the chosen DBMS type. Relational model, Object model.
 - **Physical (Low-level):** Describes actual storage. B-trees, hash files, extents.
-

Q9. What is the Relational Data Model?

A: Proposed by E.F. Codd in 1970. Data is organized as a collection of **relations (tables)**. Each relation has:

- **Relation (Table):** A set of tuples.
- **Tuple (Row):** A single record.
- **Attribute (Column):** A named property.

- **Domain:** Set of allowed values for an attribute.
- **Degree:** Number of attributes.
- **Cardinality:** Number of tuples.

The relational model has a strong mathematical foundation in **set theory** and **predicate logic**, enabling formal query languages like relational algebra.

3. Entity-Relationship (ER) Model

Q10. What is the ER Model?

A: The Entity-Relationship model is a **conceptual data model** used to represent the structure of a database graphically before implementation. Proposed by Peter Chen in 1976.

Key components:

- **Entity:** A real-world object distinguishable from others. Represented by rectangles.
Example: Student, Course, Employee.
 - **Attribute:** Property of an entity. Represented by ovals. Example: StudentID, Name, Age.
 - **Relationship:** Association between two or more entities. Represented by diamonds.
Example: Student *Enrolls* in Course.
-

Q11. What are the types of attributes in ER modeling?

A:

Type	Description	Example
Simple	Cannot be divided further	Age, StudentID
Composite	Can be divided into sub-attributes	Name → (First, Middle, Last)
Single-valued	One value per entity	StudentID
Multi-valued	Multiple values possible	Phone numbers, Hobbies
Derived	Computed from another attribute	Age derived from DateOfBirth
Null	May have no value	MiddleName
Key	Uniquely identifies an entity	StudentID (underlined in ER diagram)

Q12. What are entity types — strong vs weak?

A:

- **Strong Entity:** Has its own primary key. Can be identified independently. Example: Student(StudentID, Name).
- **Weak Entity:** Cannot be uniquely identified by its own attributes alone. Depends on a related **strong entity (owner)**. Has a **partial key** (discriminator). Represented by double rectangles. Example: OrderItem depends on Order; identified by (OrderID + ItemSeq).

The relationship between a weak entity and its owner is called an **identifying relationship** (double diamond).

Q13. What are the types of relationships and cardinalities?

A:

Cardinality Ratios:

- **1:1 (One-to-One):** Each entity in A relates to at most one in B and vice versa. Example: Person–Passport.
- **1:N (One-to-Many):** One entity in A relates to many in B. Example: Department–Employees.
- **M:N (Many-to-Many):** Many in A relate to many in B. Example: Student–Course.

Participation Constraints:

- **Total Participation** (double line): Every entity must participate. Example: Every Employee must work for a Department.
 - **Partial Participation** (single line): Participation is optional. Example: Not every Employee manages a Department.
-

Q14. What is an ER diagram? How is it converted to relational tables?

A: An ER diagram visually represents entities, attributes, and relationships.

ER to Relational Mapping Rules:

1. **Strong Entity** → Table with same attributes; primary key = entity key.
 2. **Weak Entity** → Table with partial key + owner's PK as FK; PK = composite (PartialKey + OwnerPK).
 3. **1:1 Relationship** → Add FK on either side (prefer total-participation side); or merge tables.
 4. **1:N Relationship** → Add FK on the N-side table pointing to the 1-side PK.
 5. **M:N Relationship** → Create a new **junction/bridge table** with PKs of both entities as composite PK (and FKs).
 6. **Multi-valued Attribute** → Create a new table with the entity's PK + the attribute value.
 7. **Composite Attribute** → Include individual sub-attributes in the entity table.
-

4. Relational Model

Q15. What are keys in a relational database?

A:

Key Type	Definition	Example
Super Key	Any set of attributes that uniquely identifies a tuple	{StudentID}, {StudentID, Name}, {Email}
Candidate Key	Minimal super key — no proper subset is also a super key	{StudentID}, {Email}
Primary Key (PK)	Chosen candidate key; no NULLs allowed	StudentID
Alternate Key	Candidate keys not chosen as PK	Email
Foreign Key (FK)	Attribute(s) referencing PK of another table	DepartmentID in Employee → references Department
Composite Key	PK made of multiple attributes	(OrderID, ItemID) in OrderItems
Surrogate Key	Artificial PK (auto-increment ID, UUID)	auto-increment id column
Natural Key	PK from real-world data	SSN, ISBN

Q16. What are integrity constraints in RDBMS?

A: Constraints enforce rules on data to maintain accuracy and consistency:

- **Domain Constraint:** Values must be from the attribute's defined domain (data type, range).
Age INT CHECK (Age >= 0)
- **NOT NULL Constraint:** Attribute cannot be NULL.
- **UNIQUE Constraint:** All values in a column must be distinct (NULLs allowed).
- **PRIMARY KEY Constraint:** Unique + Not Null. Identifies each row.
- **FOREIGN KEY (Referential Integrity):** FK value must exist in the referenced PK (or be NULL). Prevents orphan records.
- **CHECK Constraint:** Custom condition must evaluate to true. CHECK (Salary > 0)
- **DEFAULT Constraint:** Provides a default value if none is specified.

Q17. What is referential integrity? What are ON DELETE/UPDATE actions?

A: Referential integrity ensures that a **foreign key value always points to an existing primary key** in the referenced table.

ON DELETE / ON UPDATE actions:

Action	Behavior
RESTRICT / NO ACTION	Reject the operation if it violates referential integrity (default)
CASCADE	Automatically propagate deletion/update to referencing rows
SET NULL	Set FK to NULL when referenced row is deleted/updated
SET DEFAULT	Set FK to its default value

```
FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
ON DELETE CASCADE
ON UPDATE SET NULL
```

5. SQL — DDL, DML, DCL, TCL

Q18. What are the categories of SQL commands?

A:

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define/modify schema
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Manipulate data
DCL	Data Control Language	GRANT, REVOKE	Access control
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

Q19. Explain DDL commands with examples.

A:

```
-- CREATE: Define a new table
CREATE TABLE Employee (
    EmpID      INT          PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL,
    DeptID     INT,
    Salary     DECIMAL(10,2) DEFAULT 0.00,
    JoinDate   DATE,
    CONSTRAINT fk_dept FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
    ON DELETE SET NULL
```

```
);

-- ALTER: Modify existing table
ALTER TABLE Employee ADD COLUMN Email VARCHAR(255) UNIQUE;
ALTER TABLE Employee DROP COLUMN JoinDate;
ALTER TABLE Employee MODIFY COLUMN Salary DECIMAL(12,2);
ALTER TABLE Employee RENAME COLUMN Name TO FullName;

-- DROP: Permanently delete table and all data
DROP TABLE Employee;

-- TRUNCATE: Delete all rows but keep structure (faster than DELETE, not
rollback-able in some DBMS)
TRUNCATE TABLE Employee;

-- RENAME
RENAME TABLE Employee TO Staff;
```

Q20. Explain DML commands with examples.

A:

```
-- INSERT
INSERT INTO Employee (EmpID, Name, DeptID, Salary)
VALUES (1, 'Alice', 10, 75000.00);

-- INSERT multiple rows
INSERT INTO Employee VALUES
    (2, 'Bob', 20, 60000),
    (3, 'Carol', 10, 90000);

-- INSERT from SELECT
INSERT INTO Archive_Employee
SELECT * FROM Employee WHERE JoinDate < '2020-01-01';

-- UPDATE
UPDATE Employee SET Salary = Salary * 1.10 WHERE DeptID = 10;

-- DELETE
DELETE FROM Employee WHERE EmpID = 3;

-- SELECT
SELECT Name, Salary FROM Employee WHERE DeptID = 10 ORDER BY Salary DESC;
```

Q21. What is the difference between DELETE, TRUNCATE, and DROP?

A:

Aspect	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Selected rows	All rows	Entire table (structure + data)
WHERE clause	Yes	No	No
Rollback	Yes (transactional)	No (in most DBMS)	No
Triggers	Fires row-level triggers	Does NOT fire triggers	Does NOT fire triggers

Aspect	DELETE	TRUNCATE	DROP
Speed	Slower (logs each row)	Faster (deallocates pages)	Immediate
Identity reset	No	Yes (resets auto-increment)	N/A

Q22. What are DCL and TCL commands?

A:

```
-- DCL: GRANT privileges
GRANT SELECT, INSERT ON Employee TO 'hr_user'@'localhost';
GRANT ALL PRIVILEGES ON company_db.* TO 'admin'@'%';

-- DCL: REVOKE privileges
REVOKE INSERT ON Employee FROM 'hr_user'@'localhost';

-- TCL: COMMIT – permanently save transaction changes
COMMIT;

-- TCL: ROLLBACK – undo all changes since last COMMIT
ROLLBACK;

-- TCL: SAVEPOINT – mark a point within a transaction
SAVEPOINT sp1;
-- ... some operations ...
ROLLBACK TO SAVEPOINT sp1;  -- undo only up to sp1
RELEASE SAVEPOINT sp1;      -- remove savepoint
```

6. SQL — Queries, Joins & Subqueries

Q23. What is the complete syntax and execution order of SELECT?

A:

```
SELECT    [DISTINCT] column_list          -- 5. Project columns
FROM      table_name                     -- 1. Identify source tables
[JOIN     other_table ON condition]       -- 2. Join tables
[WHERE     condition]                    -- 3. Filter rows
[GROUP BY column_list]                   -- 4. Group rows
[HAVING    condition]                    -- 5. Filter groups
[ORDER BY column_list [ASC|DESC]]         -- 6. Sort result
[LIMIT     n OFFSET m];                  -- 7. Paginate
```

Logical execution order (not written order): FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

This is why you **cannot** use a SELECT alias in a WHERE clause (WHERE executes before SELECT).

Q24. Explain all types of SQL JOINS.

A:

```
-- Sample tables: Employee(EmpID, Name, DeptID), Department(DeptID, DeptName)

-- INNER JOIN: Returns rows where join condition is met in BOTH tables
SELECT E.Name, D.DeptName
FROM Employee E
INNER JOIN Department D ON E.DeptID = D.DeptID;

-- LEFT (OUTER) JOIN: All rows from LEFT table + matching rows from right; NULLs
for no match
SELECT E.Name, D.DeptName
FROM Employee E
LEFT JOIN Department D ON E.DeptID = D.DeptID;

-- RIGHT (OUTER) JOIN: All rows from RIGHT table + matching from left
SELECT E.Name, D.DeptName
FROM Employee E
RIGHT JOIN Department D ON E.DeptID = D.DeptID;

-- FULL OUTER JOIN: All rows from both tables; NULLs where no match
SELECT E.Name, D.DeptName
FROM Employee E
FULL OUTER JOIN Department D ON E.DeptID = D.DeptID;

-- CROSS JOIN: Cartesian product – every row in A paired with every row in B
SELECT E.Name, D.DeptName
FROM Employee E CROSS JOIN Department D;
-- Result: |Employee| × |Department| rows

-- SELF JOIN: Join a table with itself
SELECT A.Name AS Employee, B.Name AS Manager
FROM Employee A
JOIN Employee B ON A.ManagerID = B.EmpID;

-- NATURAL JOIN: Automatically joins on columns with the same name (avoid in
production)
SELECT * FROM Employee NATURAL JOIN Department;
```

Q25. What is the difference between WHERE and HAVING?

A:

- **WHERE:** Filters **individual rows** before grouping. Cannot use aggregate functions.
- **HAVING:** Filters **groups** after GROUP BY. Can use aggregate functions.

```
-- Find departments where average salary > 70000, for employees with salary >
30000
SELECT DeptID, AVG(Salary) AS AvgSalary
FROM Employee
WHERE Salary > 30000          -- Filter rows first (WHERE)
GROUP BY DeptID
HAVING AVG(Salary) > 70000;   -- Filter groups (HAVING)
```

Q26. What are subqueries? Types?

A: A subquery is a query nested inside another query.

```
-- Scalar subquery: Returns single value
SELECT Name FROM Employee
WHERE Salary = (SELECT MAX(Salary) FROM Employee);

-- Column subquery: Returns a single column
SELECT Name FROM Employee
WHERE DeptID IN (SELECT DeptID FROM Department WHERE Location = 'NYC');

-- Row subquery: Returns a single row
SELECT * FROM Employee
WHERE (DeptID, Salary) = (SELECT DeptID, MAX(Salary) FROM Employee GROUP BY
DeptID LIMIT 1);

-- Table subquery (inline view / derived table): Returns a table
SELECT D.DeptName, E.AvgSal
FROM Department D
JOIN (SELECT DeptID, AVG(Salary) AS AvgSal FROM Employee GROUP BY DeptID) E
ON D.DeptID = E.DeptID;

-- Correlated subquery: References outer query; executed once per outer row
SELECT Name FROM Employee E1
WHERE Salary > (SELECT AVG(Salary) FROM Employee E2 WHERE E2.DeptID =
E1.DeptID);
```

Q27. What is a correlated subquery vs a non-correlated subquery?

A:

- **Non-correlated:** Inner query executes **once** independently, then outer query uses the result. More efficient.
- **Correlated:** Inner query references a column from the outer query, so it executes **once per row** of the outer query. Potentially slow but more powerful.

Correlated subqueries can often be rewritten as JOINS for better performance.

Q28. What are EXISTS and NOT EXISTS?

A: EXISTS returns TRUE if the subquery returns any row(s). Commonly used with correlated subqueries.

```
-- Find departments that have at least one employee
SELECT DeptName FROM Department D
WHERE EXISTS (
    SELECT 1 FROM Employee E WHERE E.DeptID = D.DeptID
);

-- Find employees with no orders
SELECT Name FROM Employee E
WHERE NOT EXISTS (
    SELECT 1 FROM Orders O WHERE O.EmpID = E.EmpID
);
```

EXISTS is often faster than IN because it short-circuits (stops on first match). IN materializes the entire subquery result.

Q29. What are UNION, INTERSECT, and EXCEPT (MINUS)?

A: Set operators combine results of two queries. Both queries must have the same number of columns and compatible data types.

```
-- UNION: All rows from both queries, removes duplicates
SELECT Name FROM Employee
UNION
SELECT Name FROM Contractor;

-- UNION ALL: All rows including duplicates (faster – no deduplication)
SELECT Name FROM Employee
UNION ALL
SELECT Name FROM Contractor;

-- INTERSECT: Only rows that appear in BOTH result sets
SELECT DeptID FROM Employee
INTERSECT
SELECT DeptID FROM Manager;

-- EXCEPT / MINUS: Rows in first query NOT in second
SELECT DeptID FROM Department
EXCEPT
SELECT DeptID FROM Employee;  -- Departments with no employees
```

7. SQL — Aggregate Functions, Views & Stored Procedures

Q30. What are SQL aggregate functions?

A:

```
SELECT
    COUNT(*)           AS TotalRows,           -- Count all rows (including NULLs)
    COUNT(Salary)      AS NonNullSalaries,      -- Count non-NULL values
    COUNT(DISTINCT DeptID) AS UniqueDepts,      -- Count distinct values
    SUM(Salary)        AS TotalSalary,
    AVG(Salary)        AS AvgSalary,
    MIN(Salary)        AS MinSalary,
    MAX(Salary)        AS MaxSalary,
    STDDEV(Salary)     AS SalaryStdDev,
    VARIANCE(Salary)   AS SalaryVariance
FROM Employee
WHERE DeptID = 10;
```

All aggregate functions ignore NULL values (except COUNT(*)).

Q31. What are window functions?

A: Window functions perform calculations across a **set of rows related to the current row** (a "window"), without collapsing rows like GROUP BY.

```
-- ROW_NUMBER, RANK, DENSE_RANK
SELECT Name, Salary, DeptID,
    ROW_NUMBER() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS RowNum,
```

```

RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS Rank,
DENSE_RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS DenseRank
FROM Employee;

-- RANK vs DENSE_RANK: Salaries 100, 90, 90, 80
-- RANK:      1, 2, 2, 4 (gap after tie)
-- DENSE_RANK: 1, 2, 2, 3 (no gap)

-- Running total
SELECT Name, Salary,
       SUM(Salary) OVER (PARTITION BY DeptID ORDER BY EmpID) AS RunningTotal
FROM Employee;

-- LAG / LEAD: Access previous/next row
SELECT Name, Salary,
       LAG(Salary, 1) OVER (ORDER BY EmpID) AS PrevSalary,
       LEAD(Salary, 1) OVER (ORDER BY EmpID) AS NextSalary
FROM Employee;

-- NTILE: Divide rows into N buckets
SELECT Name, Salary,
       NTILE(4) OVER (ORDER BY Salary DESC) AS Quartile
FROM Employee;

```

Q32. What is a View? Advantages and limitations?

A: A view is a **virtual table** based on a SELECT query. It doesn't store data itself (usually); it stores the query definition.

```

-- Create view
CREATE VIEW HighSalaryEmployees AS
SELECT EmpID, Name, Salary, DeptID
FROM Employee
WHERE Salary > 80000;

-- Use view like a table
SELECT * FROM HighSalaryEmployees WHERE DeptID = 10;

-- Updatable views (simple cases)
UPDATE HighSalaryEmployees SET Salary = 85000 WHERE EmpID = 5;

-- Drop view
DROP VIEW HighSalaryEmployees;

```

Advantages: Simplicity, security (hide columns/rows), logical data independence, reusability.

Limitations: Non-updatable for complex views (with JOINS, GROUP BY, DISTINCT, aggregates, subqueries). No performance benefit unless materialized.

Q33. What is a Materialized View?

A: A materialized view **physically stores the result** of the query on disk and is periodically refreshed. Provides a real performance boost for expensive aggregations.

```

-- PostgreSQL syntax
CREATE MATERIALIZED VIEW DeptSalaryStats AS
SELECT DeptID, AVG(Salary) AS AvgSal, COUNT(*) AS HeadCount
FROM Employee

```

```
GROUP BY DeptID;

-- Refresh manually
REFRESH MATERIALIZED VIEW DeptSalaryStats;
```

Trade-off: Faster reads, but data may be stale between refreshes, and storage/refresh cost.

Q34. What are stored procedures and functions?

A:

Stored Procedure: Pre-compiled SQL code stored in the database. Called explicitly. Can have IN/OUT/INOUT parameters. Can perform DML, DDL, TCL.

```
DELIMITER //
CREATE PROCEDURE RaiseSalary(IN dept INT, IN raise_pct DECIMAL(5,2))
BEGIN
    UPDATE Employee
    SET Salary = Salary * (1 + raise_pct / 100)
    WHERE DeptID = dept;
    COMMIT;
END //
DELIMITER ;

CALL RaiseSalary(10, 15.0);
```

Function: Returns a single value. Used inside SQL expressions. Should be side-effect-free (no DML ideally).

```
CREATE FUNCTION GetFullName(fname VARCHAR(50), lname VARCHAR(50))
RETURNS VARCHAR(100)
DETERMINISTIC
RETURN CONCAT(fname, ' ', lname);

SELECT GetFullName(FirstName, LastName) FROM Employee;
```

Q35. What are Triggers?

A: A trigger is a stored procedure that **automatically executes** in response to a DML event (INSERT, UPDATE, DELETE) on a table.

```
CREATE TRIGGER LogSalaryChange
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary <> NEW.Salary THEN
        INSERT INTO SalaryAudit(EmpID, OldSalary, NewSalary, ChangeDate)
        VALUES (OLD.EmpID, OLD.Salary, NEW.Salary, NOW());
    END IF;
END;
```

Types: BEFORE / AFTER × INSERT / UPDATE / DELETE = 6 combinations.

Use cases: Audit logging, enforcing complex business rules, auto-computing derived columns, maintaining denormalized data.

Caution: Triggers can cause hidden side effects, debugging difficulty, and performance issues if

overused.

8. Normalization

Q36. What is normalization? Why is it needed?

A: Normalization is the process of **organizing a relational database to reduce data redundancy and improve data integrity** by decomposing large tables into smaller, well-structured ones.

Problems solved (anomalies in unnormalized tables):

- **Insertion Anomaly:** Can't add data without unrelated data. E.g., can't add a course unless a student is enrolled.
 - **Update Anomaly:** Changing one fact requires updating multiple rows; inconsistency if some are missed.
 - **Deletion Anomaly:** Deleting one fact accidentally deletes another. E.g., deleting the last student in a course deletes the course info.
-

Q37. What is Functional Dependency?

A: A functional dependency $X \rightarrow Y$ means that the value of X **uniquely determines** the value of Y . If two tuples have the same X value, they must have the same Y value.

- **Trivial FD:** $Y \subseteq X$ (e.g., $\{A,B\} \rightarrow A$). Always holds.
 - **Non-trivial FD:** $Y \not\subseteq X$. Meaningful.
 - **Fully Functional Dependency:** $X \rightarrow Y$ and no proper subset of X determines Y .
 - **Partial Dependency:** A non-key attribute depends on part of a composite key.
 - **Transitive Dependency:** $X \rightarrow Y$ and $Y \rightarrow Z$ (so $X \rightarrow Z$ transitively, where Y is not a key).
-

Q38. Explain all Normal Forms.

A:

1NF (First Normal Form):

- Each column contains **atomic (indivisible)** values.
- No repeating groups or arrays.
- Each row is unique (has a primary key).

VIOLATION: Phone = "555-1234, 555-5678" (multi-valued)

FIX: Separate row per phone number, or separate PhoneNumbers table

2NF (Second Normal Form):

- Must be in 1NF.
- **No partial dependencies:** Every non-key attribute is fully functionally dependent on the **entire** primary key (matters only for composite keys).

Table: OrderDetail(OrderID, ProductID, ProductName, Quantity)
PK: (OrderID, ProductID)
VIOLATION: ProductName depends only on ProductID (partial dependency)
FIX: Split into OrderDetail(OrderID, ProductID, Quantity)
and Product(ProductID, ProductName)

3NF (Third Normal Form):

- Must be in 2NF.
- **No transitive dependencies:** Non-key attributes must not depend on other non-key attributes.

Table: Employee(EmpID, DeptID, DeptName)
PK: EmpID
VIOLATION: DeptName depends on DeptID (non-key $\rightarrow$ non-key = transitive)
FIX: Employee(EmpID, DeptID)
Department(DeptID, DeptName)

BCNF (Boyce-Codd Normal Form):

- Stronger version of 3NF.
- For every non-trivial FD $X \rightarrow Y$, **X must be a super key**.
- 3NF allows a non-key attribute to determine part of a candidate key; BCNF does not.

Table: CourseTeacher(Student, Course, Teacher)
FDs: {Student, Course} $\rightarrow$ Teacher; Teacher $\rightarrow$ Course
VIOLATION: Teacher $\rightarrow$ Course but Teacher is not a super key
FIX: Split into (Student, Teacher) and (Teacher, Course)
[may lose some FD coverage – trade-off between BCNF and lossless/dependency preservation]

4NF (Fourth Normal Form):

- Must be in BCNF.
- **No multi-valued dependencies:** $A \twoheadrightarrow B$ means A determines a set of values for B independent of other attributes.

Table: Employee(EmpID, Skill, Language)
VIOLATION: EmpID $\twoheadrightarrow$ Skill and EmpID $\twoheadrightarrow$ Language (independent multi-valued facts)
FIX: Employee_Skill(EmpID, Skill) and Employee_Language(EmpID, Language)

5NF (Fifth Normal Form / PJNF):

- Must be in 4NF.
 - No **join dependencies** that are not implied by candidate keys. A table cannot be reconstructed from smaller projections without losing information unless the join dependency is implied by the keys.
 - Rarely applied in practice.
-

Q39. What is denormalization?

A: Denormalization is the deliberate introduction of redundancy into a normalized schema to **improve read performance** at the cost of write complexity and storage.

When to denormalize:

- Reporting/analytics queries that JOIN many tables are too slow.
- Read-heavy workloads (OLAP, data warehouses).
- Caching computed results (e.g., storing OrderTotal instead of computing SUM each time).

Trade-offs: Faster reads, but updates are more complex (must update redundant copies), and risk of inconsistency.

9. Transactions & ACID Properties

Q40. What is a transaction?

A: A transaction is a **logical unit of work** comprising one or more database operations that must be executed **atomically** — either all succeed or all fail, leaving the database in a consistent state.

```
BEGIN TRANSACTION;  
    UPDATE Account SET Balance = Balance - 500 WHERE AccID = 1;  -- Debit  
    UPDATE Account SET Balance = Balance + 500 WHERE AccID = 2;  -- Credit  
COMMIT;  -- Both succeed or ROLLBACK both
```

Without transactions, a crash between the two UPDATES would lose \$500.

Q41. What are ACID properties?

A:

A — Atomicity:

- A transaction is treated as a single unit: **all or nothing**.
- If any operation fails, all changes are rolled back.
- Implemented by: Undo logs, rollback mechanism.

C — Consistency:

- A transaction brings the database from one **valid state to another valid state**.
- All integrity constraints must be satisfied before and after the transaction.
- Responsibility shared between DBMS (constraints) and application logic.

I — Isolation:

- Concurrent transactions execute as if they are **serial** (no interference).
- Changes of an in-progress transaction are invisible to other transactions (depending on isolation level).
- Implemented by: Locks, MVCC (Multi-Version Concurrency Control).

D — Durability:

- Once a transaction is committed, its changes are **permanent** even if the system crashes.
- Implemented by: Write-ahead logging (WAL), checkpointing, transaction logs.

Q42. What are transaction states?

A:

```
Active → Partially Committed → Committed
  ↓           ↓
Failed      (on error)
  ↓
Aborted (Rolled Back)
```

- **Active:** Transaction is executing.
- **Partially Committed:** Last statement executed; awaiting commit.
- **Committed:** COMMIT issued; changes are permanent.
- **Failed:** Error occurred; rollback needed.
- **Aborted:** Rolled back; database restored to prior state.

Q43. What are transaction isolation levels?

A: Isolation levels define how visible a transaction's intermediate state is to other concurrent transactions. Higher isolation = fewer anomalies but less concurrency.

Anomalies:

- **Dirty Read:** Reading uncommitted changes of another transaction.
- **Non-repeatable Read:** Same row read twice gives different values because another transaction modified it between reads.
- **Phantom Read:** Same query returns different sets of rows because another transaction inserted/deleted rows between reads.

Isolation Level	Dirty Read	Non-repeatable	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

Most databases default to **READ COMMITTED** (PostgreSQL, Oracle) or **REPEATABLE READ** (MySQL InnoDB).

10. Concurrency Control

Q44. What is concurrency control? Why is it needed?

A: Concurrency control manages simultaneous execution of transactions to ensure **isolation** and **consistency** without sacrificing performance.

Problems without concurrency control:

- **Lost Update:** T1 and T2 both read a value, both update it — one update overwrites the other.
 - **Dirty Read:** T1 reads T2's uncommitted changes; T2 rolls back → T1 used invalid data.
 - **Non-repeatable Read:** T1 reads a row; T2 updates it; T1 reads again — different value.
 - **Phantom Read:** T1 runs a range query; T2 inserts a row in the range; T1 reruns — different rows.
-

Q45. What is locking? Types of locks?

A: Locking prevents conflicting operations on the same data.

Lock Types:

- **Shared Lock (S):** Allows multiple transactions to read simultaneously. No writes allowed while shared lock held.
- **Exclusive Lock (X):** Only one transaction can hold. No other reads or writes.

Compatibility Matrix:

	S	X
S	Compatible	Incompatible
X	Incompatible	Incompatible

Lock Granularity:

- **Row-level:** Fine granularity, high concurrency, high overhead.
- **Page-level:** Medium granularity.
- **Table-level:** Coarse granularity, low concurrency, low overhead.
- **Database-level:** Used for backups.

Intention Locks (IS, IX, SIX): Used in hierarchical locking to signal intent at a higher granularity (e.g., intent to lock rows in a table).

Q46. What is Two-Phase Locking (2PL)?

A: 2PL is a concurrency control protocol that guarantees **serializability** (the gold standard for correctness).

Two phases:

1. **Growing Phase:** Transaction can acquire locks but cannot release any.
2. **Shrinking Phase:** Transaction can release locks but cannot acquire new ones.

Once a lock is released, no new locks can be acquired.

Variants:

- **Basic 2PL:** Guarantees serializability; susceptible to deadlock.
 - **Strict 2PL:** Releases all exclusive locks only at commit/rollback. Prevents cascading rollbacks.
 - **Rigorous 2PL:** Releases ALL locks (S and X) only at commit/rollback. Simplest recovery.
-

Q47. What is MVCC (Multi-Version Concurrency Control)?

A: MVCC maintains **multiple versions of each data item** to allow readers and writers to operate concurrently without blocking each other.

How it works:

- Each write creates a **new version** of the row with a timestamp or transaction ID.
- Readers see the **latest committed version** that existed when their transaction started — they don't block writers.
- Writers create new versions — they don't block readers.

Benefits: High read concurrency, no reader-writer blocking, enables snapshot isolation.

Used by: PostgreSQL, Oracle (using undo segments), MySQL InnoDB, SQL Server (Snapshot Isolation).

11. Deadlocks

Q48. What is a deadlock?

A: A deadlock is a situation where two or more transactions are **waiting for each other** to release locks, causing all of them to wait indefinitely.

```
T1 holds lock on A, wants lock on B
T2 holds lock on B, wants lock on A
→ Both wait forever
```

Q49. What are deadlock prevention and detection strategies?

A:

Prevention — Ensure at least one deadlock condition is never met:

- **Wait-Die** (Non-preemptive): Older transaction waits; younger transaction aborts and restarts. "Old waits, young dies."
- **Wound-Wait** (Preemptive): Older transaction preempts (wounds) younger; younger waits for older. "Old wounds, young waits."
- **No Waiting**: If a lock can't be granted, transaction immediately aborts.
- **Conservative 2PL**: Acquire all needed locks before starting — prevents deadlocks but reduces concurrency.

Detection + Recovery:

- Maintain a **Wait-For Graph (WFG)**: Nodes = transactions; edges = "T1 waits for T2."
- Periodically check for **cycles** in WFG; a cycle = deadlock.
- **Victim selection**: Choose a transaction to abort (criteria: youngest, fewest changes, fewest locks held).
- Abort victim, rollback, release locks, restart.

Timeout: Abort a transaction if it waits longer than a threshold. Simple but may abort non-deadlocked transactions.

12. Indexing & Hashing

Q50. What is an index in DBMS?

A: An index is a **data structure that speeds up data retrieval** by providing quick access paths to rows without scanning the entire table. It comes at the cost of additional storage and slower writes (index must be updated on INSERT/UPDATE/DELETE).

Analogy: A book index vs reading every page.

Q51. What are the types of indexes?

A:

By Structure:

- **B-Tree Index:** Balanced tree. Supports equality, range queries, ORDER BY. Default index type in most DBMS.
- **Hash Index:** Uses hash function for O(1) equality lookups. Does NOT support range queries. Used in memory-optimized tables.
- **Bitmap Index:** Bit array per distinct value. Efficient for low-cardinality columns (e.g., Gender, Status). Great for OLAP/data warehouse; bad for high-write OLTP.
- **Full-Text Index:** Inverted index for text search (MATCH AGAINST in MySQL, tsvector in PostgreSQL).
- **Spatial Index (R-Tree):** For geographic/spatial data (PostGIS).

By Coverage:

- **Primary Index:** Built on the ordering key of a sorted file. One per table.
- **Secondary Index:** Built on non-ordering attributes. Multiple allowed.
- **Clustered Index:** The physical order of rows matches the index order. One per table. The table IS the B-tree.
- **Non-clustered Index:** Separate structure with pointers to actual rows. Multiple per table.
- **Covering Index:** Index includes all columns needed by a query — no need to access the actual table rows ("index-only scan").
- **Composite Index:** Index on multiple columns. Key: column order matters. A composite index on (A, B, C) supports queries filtering on A, (A,B), (A,B,C) but NOT B alone.
- **Unique Index:** Enforces uniqueness; implicitly created for PRIMARY KEY and UNIQUE constraints.
- **Partial/Filtered Index:** Index over a subset of rows. `CREATE INDEX idx ON Orders(Status) WHERE Status = 'Pending'.`

Q52. How does a B-Tree index work?

A: A B-Tree (Balanced Tree) is a self-balancing search tree where all leaf nodes are at the same depth.

Properties:

- All keys are sorted.
- Each node has multiple keys and child pointers (branching factor = order m).
- Search, insert, delete: O(log n).

- Leaf nodes are linked (B+ Tree variant) enabling efficient range scans.

B+ Tree (used in most DBMS):

- All data pointers are in **leaf nodes** only. Internal nodes store only keys for navigation.
- Leaf nodes are linked in a **doubly-linked list** for efficient range queries.
- Enables full table scans along the leaf level without going back to internal nodes.

Query: `SELECT * FROM Employee WHERE Salary BETWEEN 60000 AND 90000`
 B+ Tree: Navigate from root to leaf node with 60000, then scan linked list until 90000.

Q53. What is the difference between clustered and non-clustered index?

A:

Aspect	Clustered Index	Non-Clustered Index
Physical order	Table rows sorted by index key	Separate structure; rows not reordered
Count per table	Only 1	Many (SQL Server: up to 999)
Leaf nodes contain	Actual data rows	Row pointers (RIDs or clustered key)
Storage	Table IS the B-tree	Additional storage required
Performance	Faster for range queries, ORDER BY	Requires extra lookup to get row data
Primary key	Usually clustered by default	—

Q54. What is hashing in databases?

A: Hashing applies a **hash function** to a search key to directly compute the location (bucket) of a record.

Static Hashing: Fixed number of buckets. Problem: overflow when buckets fill up; waste if over-provisioned.

Dynamic Hashing (Extendible Hashing, Linear Hashing): Buckets can split/merge as data grows. Avoids fixed-size limitation.

Hash indexes are ideal for equality searches (`WHERE id = 42`) but cannot support range queries because hash values have no ordering.

13. Query Processing & Optimization

Q55. What are the steps in query processing?

A:

SQL Query

↓

1. PARSING & TRANSLATION

- Syntax check, semantic check (table/column existence)
- Translate to internal representation (relational algebra expression tree)

↓

2. QUERY OPTIMIZATION

- Generate multiple equivalent execution plans
- Estimate cost of each plan (I/O, CPU, network)
- Select the plan with minimum estimated cost

↓

3. QUERY EXECUTION

- Execute the chosen plan
- Return results

Q56. What is query optimization?

A: Query optimization finds the most efficient way to execute a query. The optimizer uses:

Heuristic (Rule-based) Optimization:

- Apply transformations that are almost always beneficial:
 - Push selections (WHERE) down the tree (reduce rows early).
 - Push projections down (reduce columns early).
 - Perform most restrictive selections first.
 - Reorder JOINS.
 - Use indexes when available.

Cost-based Optimization:

- Estimate cost using statistics: table sizes, distinct values, histograms, index availability.
- Enumerate plans (or use dynamic programming for large queries).
- Choose minimum-cost plan.

Q57. What are query execution plan and EXPLAIN?

A: The execution plan shows how the database engine will execute a query — which indexes are used, join methods, sort operations, estimated vs actual row counts.

```
EXPLAIN SELECT E.Name, D.DeptName
FROM Employee E JOIN Department D ON E.DeptID = D.DeptID
WHERE E.Salary > 80000;
```

```
-- EXPLAIN ANALYZE (PostgreSQL) actually runs the query and shows real stats
EXPLAIN ANALYZE SELECT ...;
```

Common operations in plans: Seq Scan (full table), Index Scan, Index Only Scan, Nested Loop Join, Hash Join, Merge Join, Sort, Aggregate.

14. File Organization & Storage

Q58. What are file organization methods in DBMS?

A:

Heap File (Unordered):

- Records stored in insertion order. No particular order.
- Search: $O(n)$ linear scan. Insert: $O(1)$. Delete: mark as deleted.

Sequential File (Ordered):

- Records sorted by a search key.
- Binary search possible: $O(\log n)$. Range queries efficient. Insert/delete expensive (must maintain order).

Hash File:

- Records stored in buckets by hash function on key.
- Equality search: $O(1)$ average. Range queries: not supported.

Clustered File (B+ Tree):

- Table stored as a B+ tree ordered by primary key.
- Range queries and equality both efficient.
- Most modern RDBMS (InnoDB, SQL Server) use clustered B+ tree as default table structure.

Q59. What is a buffer pool / buffer manager?

A: The **buffer pool** is a region of main memory that the DBMS uses to **cache disk pages** to avoid slow disk I/O. The buffer manager decides which pages to keep in memory and which to evict.

- **Page:** Fixed-size unit of storage (typically 4KB, 8KB, or 16KB).
- **Dirty page:** Modified in memory but not yet written to disk.
- **Replacement policies:** LRU, Clock (approximates LRU), LRU-K, MRU (better for sequential scans).
- **Write-ahead logging:** Dirty pages cannot be written before their log records are written to disk.

15. Recovery & Crash Management

Q60. What is database recovery?

A: Recovery restores the database to a **consistent state after a failure** (system crash, disk failure, power outage, transaction abort).

Types of failures:

- **Transaction failure:** Logic error, runtime error, or explicit rollback. Recovery: Undo the transaction.
 - **System failure:** OS crash, power failure. Contents of volatile memory (buffer pool) are lost. Recovery: Redo committed, undo uncommitted.
 - **Media failure:** Disk damage. Recovery: Restore from backup + replay log.
-

Q61. What is Write-Ahead Logging (WAL)?

A: WAL is the fundamental principle of database recovery:

Before any database change is written to disk, the corresponding log record must first be written to stable storage (the transaction log).

This ensures:

- **Redo:** Log has enough info to redo committed transactions.
- **Undo:** Log has enough info to undo incomplete transactions.

Every log record contains: (TransactionID, PageID, Old value, New value, LSN — Log Sequence Number).

Q62. What is the ARIES recovery algorithm?

A: ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) is the industry-standard recovery algorithm (used in DB2, SQL Server, PostgreSQL-inspired).

Three phases after a crash:

1. **Analysis Phase:** Scan log from last checkpoint forward. Determine which transactions were active at crash (need undo) and which dirty pages were in buffer pool.
2. **Redo Phase:** Replay ALL log records from the earliest dirty page LSN — redo every change (even from uncommitted transactions) to restore buffer pool to crash state.
3. **Undo Phase:** Walk log backward, undo changes of all transactions that were active (not committed) at crash time.

ARIES supports **no-force, steal** buffer management: maximum flexibility and performance.

Q63. What are checkpoints in database recovery?

A: A checkpoint is a point at which the DBMS **flushes all dirty buffer pages to disk** and records this in the log. Recovery only needs to process log records from the last checkpoint — not the entire log.

Types:

- **Sharp Checkpoint:** Pause all activity, flush all dirty pages, write checkpoint record. Simple but interrupts service.
- **Fuzzy Checkpoint:** Begin checkpoint while transactions continue. Log the start and end of checkpoint; record dirty page table and active transactions.

16. Relational Algebra & Calculus

Q64. What is Relational Algebra?

A: Relational Algebra is a **procedural query language** based on set theory and algebra that provides a formal foundation for SQL. Operations take one or two relations and produce a new relation.

Fundamental Operations:

Operation	Symbol	SQL Equivalent	Description
Select	σ	WHERE	Filter rows by condition
Project	π	SELECT (columns)	Select specific columns
Union	$\cup$	UNION	All rows from both relations
Difference	$-$	EXCEPT/MINUS	Rows in R not in S
Cartesian Product	$\times$	CROSS JOIN	All combinations
Rename	ρ	AS	Rename relation/attribute

Derived Operations:

- **Intersection:** $R \cap S = R - (R - S)$
- **Natural Join:** $\bowtie$ (join on matching attribute names)
- **Theta Join:** $\bowtie_{\theta}$ (join on condition θ)
- **Equijoin:** Theta join with equality condition
- **Division:** $R \div S$ (tuples in R that match ALL tuples in S)

Q65. What is Relational Calculus?

A: Relational Calculus is a **declarative** query language (describe WHAT, not HOW). Two variants:

Tuple Relational Calculus (TRC): $\{t \mid P(t)\}$ — Set of all tuples t satisfying predicate P .

Example: Find all employees earning > 50000 : $\{t \mid t \in \text{Employee} \wedge t.\text{Salary} >$

50000}

Domain Relational Calculus (DRC): Uses domain variables (column values) rather than tuple variables. $\{ \langle n, s \rangle \mid \langle n, _, s \rangle \in \text{Employee} \wedge s > 50000 \}$

Both TRC and DRC are **relationally complete** — expressible in relational algebra can be expressed in both calculi. Codd's theorem.

17. NoSQL Databases

Q66. What is NoSQL? When to use it over RDBMS?

A: NoSQL ("Not Only SQL") databases are non-relational databases designed for:

- **Flexible/dynamic schemas** (schema-less or schema-flexible)
- **Horizontal scalability** (scale out across commodity servers)
- **High write throughput** and massive data volumes
- **Specific data models** (documents, key-value, graphs, columns)

Use RDBMS when: Complex queries, strong consistency, ACID transactions, complex relationships, reporting/analytics.

Use NoSQL when: Huge scale, flexible data structures, high write loads, simple access patterns, real-time web apps, caching.

Q67. What are the types of NoSQL databases?

A:

Key-Value Stores:

- Data: Key $\rightarrow$ Value (blob, JSON, anything).
- Operations: GET, PUT, DELETE.
- Extremely fast for simple lookups. No complex queries.
- Examples: Redis, DynamoDB, Riak.
- Use cases: Caching, session management, shopping cart, leaderboards.

Document Databases:

- Data stored as JSON/BSON documents.
- Documents can have nested structures and arrays.
- Flexible schema: documents in same collection can have different fields.
- Examples: MongoDB, CouchDB, Firestore.
- Use cases: Content management, user profiles, product catalogs, mobile apps.

Column-Family (Wide-Column) Stores:

- Data organized by columns rather than rows. Sparse storage.
- Efficient for queries on specific columns across many rows.
- Examples: Apache Cassandra, HBase, Google Bigtable.
- Use cases: Time-series data, IoT telemetry, write-heavy analytics, log storage.

Graph Databases:

- Data as nodes (entities) and edges (relationships) with properties.
 - Optimized for traversing relationships.
 - Examples: Neo4j, Amazon Neptune, JanusGraph.
 - Use cases: Social networks, recommendation engines, fraud detection, knowledge graphs.
-

Q68. What is the CAP Theorem?

A: CAP Theorem (Brewer's Theorem) states that a distributed system can guarantee at most **two of the three** properties simultaneously:

- **C (Consistency):** Every read receives the most recent write or an error.
- **A (Availability):** Every request receives a response (not necessarily the most recent data).
- **P (Partition Tolerance):** System continues operating even if some nodes cannot communicate (network partition).

Since network partitions are inevitable in distributed systems, **you must choose between C and A** when a partition occurs:

- **CP Systems:** Sacrifice availability during partitions. Return errors rather than stale data. Example: HBase, Zookeeper, MongoDB (strong consistency mode).
 - **AP Systems:** Sacrifice consistency during partitions. Return possibly stale data. Example: Cassandra, CouchDB, DynamoDB (eventual consistency mode).
 - **CA Systems:** Not partition-tolerant — only work in single-node or tightly-coupled systems. Traditional RDBMS.
-

Q69. What is eventual consistency?

A: In an eventually consistent system, if no new updates are made to a data item, all replicas will **eventually converge** to the same value — but at any given moment, different nodes may return different values.

This is a weaker guarantee than strong consistency but enables much higher availability and performance in distributed systems.

Techniques: Read repair, anti-entropy background process, vector clocks for conflict detection and resolution.

Q70. What is the BASE model?

A: BASE is the NoSQL alternative to ACID:

- **BA (Basically Available):** The system guarantees availability according to CAP — responses always returned, possibly stale.
- **S (Soft State):** The state of the system may change over time even without new input (due to eventual consistency propagation).
- **E (Eventually Consistent):** Given time with no new updates, all nodes will converge.

BASE trades consistency for availability and performance. Most NoSQL databases follow BASE.

18. Distributed Databases

Q71. What is a distributed database?

A: A distributed database stores data across **multiple physically separate machines (nodes)** connected by a network, appearing as a single database to users.

Types:

- **Homogeneous:** All nodes run the same DBMS software.
- **Heterogeneous:** Different DBMS at different nodes; unified via middleware.
- **Federated:** Autonomous databases with a global schema.

Advantages: High availability, fault tolerance, geographic distribution, horizontal scalability. **Challenges:** Data consistency, distributed transactions, network partitions, increased complexity.

Q72. What is data replication vs data partitioning (sharding)?

A:

Replication: Same data stored on multiple nodes.

- **Goal:** High availability, fault tolerance, read scalability.
- **Types:** Master-slave (primary-replica), multi-master, synchronous vs asynchronous.

Partitioning (Sharding): Data divided across nodes — each node holds a different subset.

- **Goal:** Scale write throughput and storage horizontally.
- **Types:**
 - **Horizontal partitioning:** Different rows on different nodes (most common).
 - **Vertical partitioning:** Different columns on different nodes.

- **Hash partitioning:** $\text{shard} = \text{hash}(\text{key}) \% \text{num_shards}$. Even distribution but hard to rebalance.
- **Range partitioning:** Shard based on key range. Easy range queries but hotspot risk.
- **Consistent hashing:** Minimizes data movement when nodes are added/removed. Used in Cassandra, DynamoDB.

Q73. What is a distributed transaction? Two-Phase Commit (2PC)?

A: A distributed transaction spans multiple nodes. Ensuring ACID across nodes requires a coordination protocol.

Two-Phase Commit (2PC):

Phase 1 — **Prepare:**

1. Coordinator sends PREPARE to all participants.
2. Each participant logs the transaction to stable storage, responds YES (ready to commit) or NO (abort).

Phase 2 — **Commit/Abort:** 3. If ALL participants voted YES → Coordinator sends COMMIT. 4. If ANY participant voted NO → Coordinator sends ABORT. 5. Participants execute and ACK.

Problems with 2PC:

- **Blocking:** If coordinator crashes after Phase 1, participants wait indefinitely (blocking protocol).
- **Performance:** Two round-trips of network messages.

Three-Phase Commit (3PC): Adds a PRE - COMMIT phase to make it non-blocking, but rarely used in practice due to complexity.

Modern systems use **Saga pattern** or **Paxos/Raft** consensus instead.

19. Advanced DBMS Concepts

Q74. What is the difference between OLTP and OLAP?

A:

Aspect	OLTP	OLAP
Full form	Online Transaction Processing	Online Analytical Processing
Purpose	Day-to-day operations	Business intelligence, reporting
Queries	Simple, short, point lookups	Complex, long, aggregate-heavy
Data volume	Current operational data	Large historical data

Aspect	OLTP	OLAP
Schema	Normalized (3NF)	Denormalized (Star/Snowflake schema)
Response time	Milliseconds	Seconds to minutes
Concurrent users	Thousands	Hundreds
Update frequency	Frequent (INSERT/UPDATE/DELETE)	Batch updates (ETL pipelines)
Examples	Banking, e-commerce, ERP	Data warehouses, BI tools

Q75. What is a data warehouse?

A: A data warehouse is a **centralized repository** optimized for analysis and reporting. It integrates data from multiple operational sources, cleans it, and structures it for analytics.

ETL Process: Extract (from OLTP sources) → Transform (clean, deduplicate, aggregate) → Load (into warehouse).

Schemas used:

- **Star Schema:** Central fact table surrounded by dimension tables. Simple, fast queries.
- **Snowflake Schema:** Dimension tables are further normalized. Saves storage, more complex queries.
- **Galaxy Schema (Fact Constellation):** Multiple fact tables share dimensions.

Concepts: Dimensions (what: product, customer, time), Facts (measurements: sales_amount, quantity), Measures, Hierarchies, OLAP Cubes, Slicing, Dicing, Drill-down, Roll-up, Pivoting.

Q76. What is partitioning in SQL databases?

A: Table partitioning divides a large table into smaller **physical partitions** that can be stored and queried independently, but appear as a single table to the application.

Types:

- **Range Partitioning:** PARTITION BY RANGE (year) — Partition by value ranges.
- **List Partitioning:** PARTITION BY LIST (country) — Partition by specific values.
- **Hash Partitioning:** PARTITION BY HASH (id) — Even distribution.
- **Composite:** Combine methods (e.g., range + hash).

Benefits: Partition pruning (query only relevant partitions), faster maintenance (drop entire partition instead of DELETE), parallel query execution.

Q77. What is a cursor in SQL?

A: A cursor provides a mechanism to **traverse query results row-by-row** — useful when set-based SQL can't solve the problem.

```
DECLARE emp_cursor CURSOR FOR
    SELECT EmpID, Salary FROM Employee WHERE DeptID = 10;

OPEN emp_cursor;

FETCH NEXT FROM emp_cursor INTO @id, @salary;
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Process each row
    FETCH NEXT FROM emp_cursor INTO @id, @salary;
END;

CLOSE emp_cursor;
DEALLOCATE emp_cursor;
```

Caution: Cursors are slow (row-by-row vs set-based operations). Always try to rewrite with set-based SQL first.

Q78. What is the difference between a primary key and a unique key?

A:

Aspect	Primary Key	Unique Key
NULLs	NOT allowed	Allowed (one or more NULLs depending on DBMS)
Per table	Only one	Multiple allowed
Index	Clustered index (usually)	Non-clustered unique index
Purpose	Row identity	Uniqueness constraint on alternate keys
Implicit index	Always	Yes, unique index created automatically

Q79. What is a surrogate key vs natural key?

A:

- **Natural Key:** A key that has business meaning — derived from real-world data. Example: SSN, ISBN, Email.
 - Pros: Meaningful, no extra column needed.
 - Cons: May change (SSN reassigned?), may be long/complex, privacy concerns.
- **Surrogate Key:** An artificial key with no business meaning, auto-generated. Example: `AUTO_INCREMENT id`, UUID.
 - Pros: Stable, compact, no privacy leak, simple joins.

- **Cons:** No inherent meaning; requires separate unique constraints for natural keys.

Most production systems use surrogate keys as PKs and add UNIQUE constraints on natural keys.

Q80. What is database sharding vs vertical scaling vs horizontal scaling?

A:

- **Vertical Scaling (Scale Up):** Add more CPU/RAM/storage to a single server. Simple but has a hardware ceiling and single point of failure.
- **Horizontal Scaling (Scale Out):** Add more servers. Distribute load. More complex but theoretically unlimited.
- **Sharding:** A form of horizontal scaling where data is partitioned across multiple database instances (shards). Each shard handles a subset of data.

Sharding challenges: Cross-shard queries, distributed transactions, rebalancing shards, hotspots, application complexity.

Q81. What is Connection Pooling?

A: Creating a new database connection for every request is expensive (TCP handshake, authentication, resource allocation). **Connection pooling** maintains a pool of pre-established connections that are reused.

Workflow:

1. Application requests a connection from the pool.
2. Pool provides an idle connection (or creates a new one up to max pool size).
3. Application uses and returns the connection to the pool.
4. Connection is reset and made available for the next request.

Tools: HikariCP (Java), pgBouncer (PostgreSQL), ProxySQL (MySQL).

Q82. What is an ORM (Object-Relational Mapper)?

A: An ORM maps database tables to programming language objects, allowing developers to interact with the database using object-oriented code instead of raw SQL.

```
# Without ORM (raw SQL)
cursor.execute("SELECT * FROM Employee WHERE DeptID = %s", (10,))

# With ORM (SQLAlchemy)
employees = session.query(Employee).filter(Employee.dept_id == 10).all()
```

Pros: Less boilerplate, type safety, database portability, built-in protection against SQL injection. **Cons:** N+1 query problem, less control over SQL, can generate inefficient queries; harder to optimize.

20. Database Design & Best Practices

Q83. What is the N+1 query problem?

A: N+1 occurs when code fetches a list of N records, then executes 1 additional query per record to fetch associated data — resulting in N+1 total queries instead of 1.

```
# N+1 Problem: 1 query for all employees + N queries for each employee's
department
employees = db.query("SELECT * FROM Employee")    # 1 query
for emp in employees:
    dept = db.query(f"SELECT * FROM Department WHERE id = {emp.dept_id}") # N
queries

# Solution: Eager loading / JOIN
employees = db.query("""
    SELECT E.*, D.DeptName
    FROM Employee E JOIN Department D ON E.DeptID = D.DeptID
""") # 1 query
```

Q84. What is SQL injection and how do you prevent it?

A: SQL injection is an attack where malicious SQL is inserted into an input field, manipulating the query.

```
-- Vulnerable:
query = "SELECT * FROM Users WHERE username = '" + input + "'";
-- Input: ' OR '1'='1
-- Resulting query: SELECT * FROM Users WHERE username = '' OR '1'='1' ←
returns all users!
```

Prevention:

1. **Parameterized queries / Prepared statements:** `SELECT * FROM Users WHERE username = ?` — input is never interpreted as SQL.
2. **ORM:** Uses parameterized queries internally.
3. **Input validation:** Whitelist allowed characters.
4. **Least privilege:** DB user should only have necessary permissions.
5. **WAF (Web Application Firewall):** Detect and block common injection patterns.

Q85. What is database indexing best practices?

A:

Index candidates:

- Columns frequently used in WHERE, JOIN ON, ORDER BY, GROUP BY.
- Foreign key columns (always index them).
- High-cardinality columns (many distinct values).

Avoid indexing:

- Columns rarely queried.
- Very low cardinality (e.g., boolean flag — bitmap index is better).
- Columns in very small tables (full scan is faster).
- Columns that are frequently updated (index maintenance overhead).

Composite index tips:

- Put most selective column first.
- Consider query patterns: (A, B) index helps WHERE A=? and WHERE A=? AND B=? but NOT WHERE B=? alone.
- Use covering indexes to avoid table lookups.

Monitor: Use EXPLAIN/EXPLAIN ANALYZE; identify slow queries; look for Seq Scans on large tables.

Q86. What is database partitioning best practices?

A:

- Use range partitioning on time columns for time-series data (partition by month/year).
 - Ensure queries include the partition key in WHERE clause to enable partition pruning.
 - Balance partition sizes — avoid hotspots (all writes to one partition).
 - Consider maintenance: Can drop old partitions instead of DELETE (much faster).
 - Test queries with EXPLAIN to confirm partition pruning is happening.
-

Q87. What is the difference between optimistic and pessimistic locking?

A:

Pessimistic Locking: Assumes conflicts WILL happen. Lock the data before reading to prevent others from modifying it.

```
SELECT * FROM Account WHERE AccID = 1 FOR UPDATE; -- Locks the row
```

- Good for: High-contention environments, write-heavy workloads.
- Drawback: Reduces concurrency; risk of deadlocks.

Optimistic Locking: Assumes conflicts are RARE. Don't lock; check at commit time if data was modified.

```
UPDATE Account SET Balance = 900, Version = 6  
WHERE AccID = 1 AND Version = 5; -- Fails if another transaction already  
updated
```

- Good for: Read-heavy, low-contention, web applications.

- Drawback: Retry logic needed on conflict.
-

Q88. What are common SQL performance tuning techniques?

A:

1. **Use indexes appropriately:** Index WHERE/JOIN/ORDER BY columns; cover with composite indexes.
 2. ****Avoid SELECT **:** Fetch only needed columns.
 3. **Use EXPLAIN:** Identify Seq Scans, missing indexes, bad join orders.
 4. **Rewrite correlated subqueries as JOINS.**
 5. **Avoid functions on indexed columns in WHERE:** WHERE YEAR(date_col) = 2024 disables index. Use WHERE date_col BETWEEN '2024-01-01' AND '2024-12-31'.
 6. **Use EXISTS instead of COUNT for existence checks.**
 7. **Limit result sets:** Use LIMIT/pagination.
 8. **Partitioning for large tables.**
 9. **Connection pooling:** Avoid connection overhead.
 10. **Denormalize hot read paths:** Materialized views, summary tables.
 11. **Batch writes:** INSERT multiple rows at once vs one-by-one.
 12. **Analyze and vacuum:** Keep statistics fresh (PostgreSQL: ANALYZE; MySQL: ANALYZE TABLE).
-

Q89. What is database replication?

A: Replication keeps copies of data synchronized across multiple database servers.

Types:

- **Master-Slave (Primary-Replica):** Writes go to master; replicated asynchronously or synchronously to slaves. Reads can be load-balanced across slaves.
- **Master-Master (Multi-Primary):** Writes accepted on any node. Requires conflict resolution. More complex.
- **Synchronous Replication:** Write is committed only after all replicas confirm. Strong consistency, higher latency.
- **Asynchronous Replication:** Write committed at primary immediately; replicas may lag. Higher performance, eventual consistency.
- **Semi-synchronous:** At least one replica confirms before commit.

Use cases: Read scalability, high availability, geographic distribution, disaster recovery.

Q90. What are some differences between MySQL, PostgreSQL, and Oracle?

A:

Feature	MySQL	PostgreSQL	Oracle
License	Open Source (GPL)	Open Source	Commercial
ACID	Yes (InnoDB)	Yes	Yes
Default isolation	Repeatable Read	Read Committed	Read Committed
JSON support	Yes (5.7+)	Advanced (JSONB)	Yes (12c+)
Full-text search	Basic	Advanced (tsvector)	Advanced
Window functions	8.0+	Full support	Full support
CTEs	8.0+	Full (recursive)	Full
Partitioning	Yes	Yes (declarative)	Yes (advanced)
Replication	Master-Slave, Group	Streaming, Logical	Advanced (Data Guard)
MVCC	Yes (InnoDB)	Yes (heap + MVCC)	Yes (undo segments)
Custom data types	Limited	Extensible	Yes
Stored procedures	Yes	PL/pgSQL + others	PL/SQL
Use case	Web apps, LAMP	Complex queries, GIS	Enterprise, ERP

☐ Quick Reference Cheat Sheet

Core Concepts

Concept	Key Point
DBMS vs File System	Concurrency, integrity, independence, recovery
Three schema levels	Physical → Logical → View
ER → Relational	M:N → junction table; 1:N → FK on N side
Keys	PK: unique + not null; FK: references PK; CK: minimal superkey
Integrity constraints	Domain, NOT NULL, UNIQUE, PK, FK, CHECK

Normalization

Normal Form	Eliminates
1NF	Non-atomic values, repeating groups
2NF	Partial dependencies (on part of composite PK)
3NF	Transitive dependencies (non-key → non-key)

Normal Form	Eliminates
BCNF	Non-superkey determinants
4NF	Multi-valued dependencies

Transactions

Concept	Key Point
ACID	Atomicity, Consistency, Isolation, Durability
Isolation levels	READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE
Dirty read	Read uncommitted data
Non-repeatable read	Same row, different value on re-read
Phantom read	Same query, different set of rows on re-run
2PL	Growing phase + Shrinking phase → Serializability
MVCC	Multiple row versions → readers don't block writers
WAL	Log before write → enables redo/undo

Indexing

Type	Best for
B+ Tree	Range queries, equality, ORDER BY, default
Hash	Equality only, O(1)
Bitmap	Low-cardinality, OLAP
Full-text	Text search
Composite	Multi-column queries (order matters!)
Covering	Index-only scan, avoid table lookup

NoSQL & Distributed

Concept	Key Point
CAP Theorem	Can only guarantee 2 of: Consistency, Availability, Partition Tolerance
BASE	Basically Available, Soft State, Eventually Consistent
ACID vs BASE	Strict consistency vs High availability
Sharding	Horizontal partition across nodes

Concept	Key Point
2PC	Prepare → Commit; blocking protocol
OLTP vs OLAP	Transactions vs Analytics

SQL Quick Reference

Command	Category
CREATE, ALTER, DROP, TRUNCATE	DDL
SELECT, INSERT, UPDATE, DELETE	DML
GRANT, REVOKE	DCL
COMMIT, ROLLBACK, SAVEPOINT	TCL
WHERE (rows) vs HAVING (groups)	Filtering
INNER, LEFT, RIGHT, FULL, CROSS, SELF	JOIN types
EXISTS vs IN	EXISTS: short-circuit; IN: materializes
DELETE vs TRUNCATE vs DROP	Row/All rows/Table

Best of luck with your DBMS interviews!